Отчёт по лабораторной работе — Neo4j

Предмет: Базы Данных
Тема: Графовая СУБД Neo4j
Дата: Февраль 2026

Описание проекта

Проект представляет собой новостной агрегатор. В рамках предыдущих лабораторных работ уже были реализованы PostgreSQL (с репликацией), MongoDB (с шардированием), Redis и мониторинг на базе Grafana+Prometheus. В данной работе к проекту добавлена графовая СУБД Neo4j.

Модель данных

Выбрана следующая модель графа на основе предметной области новостного агрегатора:

Узлы (5 типов):

- `Article` — статья (title, category, views, likes, shares, publishDate)
- `Author` — автор (name, email, rating, country)
- `Source` — источник новостей (name, website, country, reliability)
- `Category` — категория (name, description, featured)
- `Tag` — тег (name, color)

Связи (5 типов):

- `(Author)-[:WROTE]->(Article)` — автор написал статью
- `(Article)-[:PUBLISHED_IN]->(Source)` — статья опубликована в источнике
- `(Article)-[:IN_CATEGORY]->(Category)` — статья относится к категории
- `(Article)-[:HAS_TAG]->(Tag)` — статья имеет тег
- `(Author)-[:WORKS_FOR]->(Source)` — автор работает в источнике

Итоговая статистика графа:

- Узлов: 105
- Связей: 275
- Свойств: 20 уникальных ключей

Работа с данными

П.1 — Минимум 3 узла и 2 связи ✅

Реализовано 5 типов узлов и 5 типов связей (см. модель выше).

П.2 — Создание данных через MERGE ✅

Все данные загружены через Python-скрипт `neo4j/scripts/seed.py` с использованием оператора `MERGE` вместо `CREATE`. MERGE гарантирует что дублирующиеся узлы не создаются — если узел уже существует, он просто обновляется.

Итого создано:

- 50 статей, 25 авторов, 7 категорий, 6 источников, 17 тегов
- 275 связей с свойствами (например, `WROTE` содержит `publishedAt`, `WORKS_FOR` содержит `since` и `role`)

П.3 — 6 запросов с фильтрацией ✅

Реализованы в `neo4j/scripts/queries.py`:

1. Статьи категории technology с фильтрацией по связи `IN_CATEGORY`
2. Авторы из USA с рейтингом > 8.0 — фильтрация по свойствам узла
3. Статьи с тегом 'ai' — фильтрация по связи `HAS_TAG`
4. Источники с надёжностью 9 — фильтрация по свойству узла
5. Статьи опубликованные в Reuters — фильтрация по связи `PUBLISHED_IN`
6. Авторы работающие в источниках из UK — фильтрация по свойству соседнего узла

П.4 — Цепочки связей и переменная длина пути ✅

Запрос с фиксированной цепочкой — авторы связанные через общие теги:

```cypher
MATCH (a1:Author)-[:WROTE]->(art1:Article)-[:HAS_TAG]->(t:Tag)
      <-[:HAS_TAG]-(art2:Article)<-[:WROTE]-(a2:Author)
WHERE a1 <> a2
RETURN DISTINCT a1.name, a2.name, t.name AS common_tag
```

Запрос с переменной длиной пути `[*2..4]`:

```cypher
MATCH path = (a:Author)-[*2..4]->(t:Tag)
RETURN a.name, t.name, length(path) AS steps
```

Этот запрос находит косвенные связи — от автора до тега через 2, 3 или 4 шага. Например: Author → Article → Tag (2 шага).

П.5 — Агрегационные запросы ✅

Реализованы 5 агрегационных запросов:

1. `COUNT + ORDER BY` — количество статей по категориям
2. `AVG + LIMIT` — топ-5 источников по среднему рейтингу авторов
3. `COLLECT + COUNT` — теги и список статей с этим тегом
4. `SUM` — источники и суммарные просмотры их статей
5. `AVG + WHERE` — категории со средними просмотрами > 20000

П.6 — Сравнение с Postgres/Mongo ✅

Реализовано в `neo4j/scripts/postgres_vs_cypher.py`. Взяты реальные запросы из файла `queries_with_joined_tables.sql` проекта и переписаны на Cypher.

Пример 1 — авторы и статистика (2 таблицы в SQL):

```sql
-- PostgreSQL
SELECT a.first_name, COUNT(n.id), AVG(n.views_count)
FROM authors a LEFT JOIN news n ON a.id = n.author_id
GROUP BY a.id ORDER BY articles_count DESC LIMIT 10;
```

```cypher
-- Cypher
MATCH (a:Author)-[:WROTE]->(art:Article)
RETURN a.name, count(art) AS articles_count, avg(art.views) AS avg_views
ORDER BY articles_count DESC LIMIT 10
```

Пример 2 — полная информация (4 таблицы + LEFT JOIN в SQL):

```sql
-- PostgreSQL — требует 4 JOIN и STRING_AGG для тегов
SELECT n.title, c.name, s.name, a.first_name, STRING_AGG(t.name, ', ')
FROM news n JOIN categories c ... JOIN sources s ... JOIN authors a ...
LEFT JOIN news_tags nt ... LEFT JOIN tags t ...
GROUP BY n.id, n.title, ... ORDER BY n.views_count DESC LIMIT 10;
```

```cypher
```

```
-- Cypher — читается как описание графа
MATCH (a:Author)-[:WROTE]->(art:Article)-[:IN_CATEGORY]->(c:Category),
    (art)-[:PUBLISHED_IN]->(s:Source)
OPTIONAL MATCH (art)-[:HAS_TAG]->(t:Tag)
RETURN art.title, c.name, s.name, a.name, collect(DISTINCT t.name) AS tags
ORDER BY art.views DESC LIMIT 10
```

Вывод: Cypher значительно проще и нагляднее для запросов с множеством связей. SQL требует явного указания ключей JOIN и громоздкого GROUP BY. В Cypher связи уже встроены в граф — запрос читается почти как обычный текст.

П.7 — Поиск общих соседей ✅

```cypher
MATCH (a1:Author)-[:WROTE]->(art1:Article)-[:HAS_TAG]->(t:Tag)
    <-[:HAS_TAG]-(art2:Article)<-[:WROTE]-(a2:Author)
WHERE a1 <> a2
RETURN a1.name, a2.name, count(DISTINCT t) AS common_tags
ORDER BY common_tags DESC LIMIT 10
```

Результат: Author_2 и Author_16 имеют максимум — 4 общих тега.

П.8 — Комбинированный запрос ✅

```cypher
MATCH (a:Author)-[:WROTE]->(art:Article)-[:PUBLISHED_IN]->(s:Source)
WHERE a.rating > 7.0 AND art.views > 15000
RETURN s.name, count(art) AS articles, avg(art.views) AS avg_views,
    collect(DISTINCT a.name) AS authors
ORDER BY avg_views DESC LIMIT 5
```

Запрос объединяет: цепочку связей (автор→статья→источник), фильтрацию по свойствам (rating, views), агрегацию (count, avg, collect) и сортировку с ограничением.

Какие задачи лучше решаются в Neo4j, нежели в Postgres/Mongo?

- Поиск косвенных связей (друзья друзей, цепочки через несколько узлов)
- Рекомендательные системы (кто ещё читал то что ты читал)
- Поиск кратчайшего пути между сущностями
- Запросы с множеством JOIN — в Cypher это просто цепочка связей
- Анализ социальных графов, графов зависимостей

Какие задачи решаются хуже?

- Агрегация больших объёмов числовых данных (SUM, AVG по миллионам строк) — Postgres быстрее
- Хранение документов с произвольной структурой — MongoDB лучше
- Простые CRUD операции с табличными данными — Postgres проще
- Полнотекстовый поиск по содержимому статей — Elasticsearch/MongoDB лучше

Где в проекте логично использовать Neo4j?

- Рекомендации статей: "читатели этой статьи также читали..."
- Поиск авторов со схожей тематикой
- Граф связей между источниками — какие источники пишут об одних темах
- Анализ популярности тегов и их взаимосвязей

Администрирование

П.1 — Развёртывание в Docker ✅

Neo4j развёрнут как отдельный сервис в существующем `docker-compose.yml` проекта.

Настройки:

- Образ: `neo4j:5.18-community`
- Порты: 7474 (Browser), 7687 (Bolt)
- Аутентификация: `neo4j/password123`
- Плагины: APOC
- Память: heap 512MB-1GB, pagecache 512MB
- Volumes: `neo4j_data` (данные), `neo4j_logs` (логи), `./neo4j/import` (импорт файлов)

Структура директории проекта:

```
neo4j/
├── import/          # CSV файлы для LOAD CSV
│   └── tags.csv
├── scripts/         # Все скрипты
│   ├── seed.py      # Загрузка данных
│   ├── queries.py   # Все запросы по ТЗ
│   ├── postgres_vs_cypher.py  # Сравнение с SQL
│   └── monitoring.py          # Мониторинг
├── backup/          # Резервные копии
│   └── neo4j.dump
└── prometheus.yml   # Конфиг мониторинга
```

П.2 — Пользователи с ролями ✅

Созданы пользователи `reader` и `publisher`:

```cypher
CREATE USER reader SET PASSWORD 'reader123' SET PASSWORD CHANGE NOT REQUIRED;
CREATE USER publisher SET PASSWORD 'publisher123' SET PASSWORD CHANGE NOT REQUIRED;
```

Примечание: Назначение ролей через `GRANT ROLE` недоступно в Community Edition Neo4j — это ограничение бесплатной версии. В Enterprise Edition или Neo4j Aura роли назначаются полноценно. Пользователи созданы и видны через `SHOW USERS`.

П.3 — Импорт через LOAD CSV ✅

Создан файл `neo4j/import/tags.csv`, скопирован в контейнер и загружен командой:

```cypher
LOAD CSV WITH HEADERS FROM 'file:///tags.csv' AS row
MERGE (t:Tag {name: row.name})
SET t.color = row.color;
```

Результат: обновлены свойства 5 тегов (добавлено поле `color`).

П.4 — Ограничения и индексы ✅

Созданы ограничения уникальности:

```cypher
CREATE CONSTRAINT article_id_unique FOR (a:Article) REQUIRE a.id IS UNIQUE;
CREATE CONSTRAINT author_email_unique FOR (a:Author) REQUIRE a.email IS UNIQUE;
CREATE CONSTRAINT category_name_unique FOR (c:Category) REQUIRE c.name IS UNIQUE;
```

Созданы индексы:

```cypher
CREATE INDEX article_views FOR (a:Article) ON (a.views);
CREATE INDEX author_rating FOR (a:Author) ON (a.rating);
```

Влияние на запросы: Проверено через EXPLAIN и PROFILE. Neo4j использует `NodeIndexSeekByRange` при поиске статей по views — вместо полного перебора узлов. Запрос `WHERE a.views > 30000` нашёл 32 статьи за 33 обращения к базе.

Другие типы ограничений в Neo4j: NODE KEY (уникальность + существование), IS NOT NULL (существование свойства) — доступны в Enterprise Edition.

П.5 — Мониторинг ✅

Реализован на двух уровнях:

1. Встроенный мониторинг через скрипт `monitoring.py`:

- Статистика графа (узлы, связи по типам)
- Настройки памяти
- Активные транзакции
- Состояние индексов и constraints
- Топ статей и авторов
- Состояние баз данных

2. Инфраструктурный мониторинг — Grafana (порт 3001) и Prometheus (порт 9090) подняты и подключены друг к другу в рамках общего docker-compose проекта.

П.6 — Резервное копирование ✅

Выполнен полный дамп базы данных:

```bash
# Остановить базу
docker exec bd-neo4j neo4j stop

# Создать дамп
docker exec bd-neo4j neo4j-admin database dump neo4j --to-path=/tmp/

# Скопировать на локальную машину
docker cp bd-neo4j:/tmp/neo4j.dump neo4j/backup/neo4j.dump

# Запустить базу обратно
docker exec bd-neo4j neo4j start
```

Дамп: 41 файл, 258MB обработано, сохранён в `neo4j/backup/neo4j.dump`.

Для восстановления используется команда:

```bash
neo4j-admin database load neo4j --from-path=/tmp/ --overwrite-destination
```

П.7 — Производительность ✅

Проверены настройки памяти:

- `server.memory.heap.initial_size`: 512MB

- `server.memory.heap.max_size`: 1GB
- `server.memory.pagecache.size`: 512MB

Использованы EXPLAIN и PROFILE для анализа запросов. EXPLAIN показывает план без выполнения, PROFILE — реальную статистику. Оба подтвердили использование индекса `article_views` при фильтрации по просмотрам.

П.8 — Отказоустойчивость ✅

Смоделирован отказ:

```bash
docker stop bd-neo4j  # база недоступна
python3 monitoring.py  # ошибка подключения
docker start bd-neo4j  # восстановление
python3 monitoring.py  # всё работает, данные на месте
```

Режим Standalone (текущий):

- Один узел, простая настройка
- При отказе — полная недоступность до перезапуска
- Подходит для: разработки, учебных проектов, небольших приложений с низкими требованиями к доступности
- Данные сохраняются в volume — при перезапуске контейнера не теряются

Когда нужен кластер (Causal Cluster):

- Требования к высокой доступности (SLA 99.9%+)
- Большие нагрузки на чтение — реплики берут на себя read-запросы
- Production-системы где даунтайм недопустим
- Минимум: 3 узла (1 leader + 2 follower)

Риски при росте графа:

- Память: граф должен помещаться в pagecache для быстрой работы. При росте нужно увеличивать `server.memory.pagecache.size`
- Запросы с переменной длиной пути `[*]` без ограничений могут обходить весь граф — нужно всегда указывать `[*min..max]`
- Шардирование графа сложнее чем шардирование таблиц — связи между узлами могут быть на разных шардах
- В Community Edition нет встроенного кластера — нужна Enterprise Edition

---

Итог

Все пункты задания выполнены. Neo4j органично вписан в существующий проект как дополнительная СУБД для работы со связями между сущностями новостного агрегатора.